

Data Preprocessing & EDA — Notes

A structured guide to understanding a dataset and performing Exploratory Data Analysis (EDA) using pandas, seaborn, and matplotlib.

1. Understanding the Dataset

Before any analysis or modeling, it is important to explore the dataset to understand its structure, quality, and content. The following steps and pandas functions help with this.

1.1 How big is the data?

Check the number of rows and columns in the dataset.

```
df.shape
```

1.2 What does the data look like?

View a quick preview of the dataset.

```
df.head()      # first 5 rows
df.sample()     # a random row
```

1.3 What is the data type of each column?

Check column names, data types, and non-null counts.

```
df.info()
```

1.4 Are there any missing values?

Count the number of missing (null) values in each column.

```
df.isnull().sum()
```

1.5 How does the data look mathematically?

Generate summary statistics (mean, min, max, standard deviation, quartiles, etc.) for numerical columns.

```
df.describe()
```

1.6 Are there duplicate values?

Check for duplicate rows in the dataset.

```
df.duplicated().sum()
```

1.7 What is the correlation between columns?

Check the correlation between numerical columns.

```
df.corr()
```

Note: To check the correlation of a single column with all others, use: `df.corr()['column_name']`

1.8 Important pandas method: Cross Tabulation (Crosstab)

A crosstab is a table that shows the relationship between two categorical columns. It counts how many times each combination of category values occurs together.

Example using the Titanic dataset:

```
pd.crosstab(titanic['Pclass'], titanic['Survived'])
```

Sample illustration of how the data might look:

Gender	Survived
Male	No
Female	Yes
Male	Yes
Female	Yes
Male	No

2. Exploratory Data Analysis (EDA)

EDA is the process of analyzing datasets to summarize their main characteristics, often using visual methods. It is divided into Univariate, Bivariate, and Multivariate analysis.

2.1 Univariate Analysis

Analyzing only one variable (one column) at a time.

- "Uni" = One
- "Variable" = Column

Purpose:

- Understand the distribution of data
- Find minimum and maximum values
- Calculate the mean (average)
- Identify missing values
- Detect outliers

Visualization for Categorical Data

If a column is categorical, the following visualizations can be used:

1. Count plot — shows the frequency of each category.

```
sns.countplot(x=df['column_name'])
```

2. Pie chart — shows the proportion of each category.

```
df['column_name'].value_counts().plot(kind='pie', autopct='%0.2f')
```

Visualization for Numerical Data

3. Histogram — a chart used to display the distribution and frequency of numerical data.

```
plt.hist(df['Age'])
```

```
plt.hist(df['Age'], bins=5)
```

4. Distribution plot (distplot) — from Seaborn, represents the univariate distribution of continuous data by combining a histogram with a Kernel Density Estimate (KDE) line.

```
sns.distplot(df['Age'])
```

5. Box plot (box-and-whisker plot) — a standardized graphical tool for displaying the distribution of numerical data based on the five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. It highlights outliers, skewness, and spread (Interquartile Range — IQR) in a compact, easy-to-read format.

```
sns.boxplot(df['column_name'])
```

Note: Box plots are best used when the data is noisy and contains outliers.

Understanding a Column Using Summary Functions

```
df['Age'].min()
```

```
df['Age'].max()
```

```
df['Age'].mean()
```

```
df['Age'].skew()
```

Interpreting skewness values:

- Skewness ≈ 0 (e.g., 0.32) $\rightarrow$ the data is approximately symmetrical.
- Skewness is positive and large (e.g., 1.23) $\rightarrow$ the data is right-skewed (the tail extends to the right; most values are concentrated on the left).

- Skewness is negative (e.g., -1.21) → the data is left-skewed (the tail extends to the left; most values are concentrated on the right).

2.2 Bivariate Analysis

Analyzing the relationship between two variables (two columns).

- "Bi" = Two
- "Variable" = Columns

Purpose:

- Understand how one variable affects another
- Determine whether a relationship exists between the two variables
- Determine whether the relationship is positive or negative

i. Scatter Plot (Numerical – Numerical)

Example using the built-in 'tips' dataset from Seaborn:

```
sns.scatterplot(x=tips['total_bill'], y=tips['tip'])
```

ii. Bar Plot (Numerical – Categorical)

Example using the Titanic dataset from Kaggle:

```
sns.barplot(x=titanic_df['Pclass'], y=titanic_df['Fare'])
sns.barplot(x=titanic_df['Pclass'], y=titanic_df['Age'])
```

iii. Box Plot (Numerical – Categorical)

Example using the Titanic dataset:

```
sns.boxplot(x=titanic_df['Sex'], y=titanic_df['Age'])
```

Note: This gives the five-number summary of one column grouped by the categories of another column.

iv. Distribution Plot (Numerical – Categorical)

Example using the Titanic dataset:

```
sns.distplot(titanic_df[titanic_df['Survived']==0]['Age'], hist=False)
sns.distplot(titanic_df[titanic_df['Survived']==1]['Age'], hist=False)
```

v. Heatmap (Categorical – Categorical)

Example using the Titanic dataset:

```
sns.heatmap(pd.crosstab(titanic['Pclass'], titanic['Survived']))
titanic_df.groupby('Pclass')['Survived'].mean() * 100
```

Note: Whenever you use groupby(), read it as: 'Make groups, then calculate something for each group.'

vi. Cluster Map (Categorical – Categorical)

```
sns.clustermap(pd.crosstab(titanic['Pclass'], titanic['Survived']))
```

vii. Pair Plot (Numerical – Numerical)

A pair plot is a visualization that shows the relationships between all numerical columns in a dataset at the same time.

```
sns.pairplot(df, hue='categorical_column_name')
```

viii. Line Plot (Numerical – Numerical)

A line plot is used to see how a value changes over time or in a specific order.

```
sns.lineplot(x='Year', y='Sales', data=df)
```

2.3 Multivariate Analysis

Multivariate analysis (MVA) is a set of statistical techniques used to analyze datasets containing more than two variables simultaneously, allowing for the examination of complex, interrelated relationships.

Example using the built-in 'tips' dataset from Seaborn:

i. Scatter Plot (Numerical – Numerical)

```
sns.scatterplot(x=tips['total_bill'], y=tips['tip'], hue=tips['sex'],  
style=tips['smoker'], size=tips['size'], data=tips)
```

ii. Bar Plot (Numerical – Categorical)

Example using the Titanic dataset from Kaggle:

```
sns.barplot(x=titanic_df['Pclass'], y=titanic_df['Fare'],  
hue=titanic_df['Sex'])
```

iii. Box Plot (Numerical – Categorical)

```
sns.boxplot(x=titanic_df['Sex'], y=titanic_df['Age'],  
hue=titanic_df['Survived'])
```

Note: This gives the five-number summary of one column grouped by two categorical columns.

iv. Pair Plot (Numerical – Numerical – Categorical)

A pair plot shows the relationships between all numerical columns in a dataset, with points colored by a categorical column.

```
sns.pairplot(df, hue='categorical_column_name')
```

v. Heatmap and Pivot Table

Example using the flights dataset:

```
sns.heatmap(flights_df.pivot_table(values='passengers', index='month',  
columns='year'))
```

vi. Cluster Map and Pivot Table

```
sns.clustermap(flights_df.pivot_table(values='passengers', index='month',  
columns='year'))
```